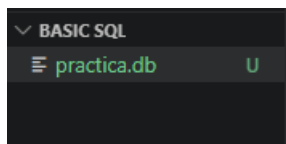
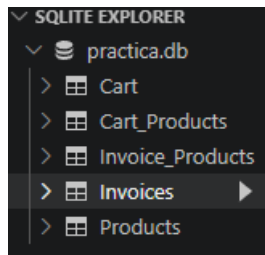


1. Cree una nueva base de datos en SQLite.



2. Replique las tablas creadas anteriormente en [Ejercicio de Bases de Datos](#), con sus respectivos PKs, FKs, constraints, y demás requerimientos.



1. Investigue cómo hacer que los PKs se generen **automáticamente**.
Para que esto suceda se digita el siguiente código INTEGER PRIMARY KEY AUTOINCREMENT, esto genera el ID automáticamente, nunca reutiliza IDs eliminados, siempre sube.
2. Utilice los tipos de datos adecuados.

SQL ▾

SELECT * FROM Invoice_Products;

id	Invoice_id	Product_id	Quantity	Total_Amount
1	1	1	2	21.98
2	2	2	5	36.6
3	3	3	2	20.6
4	2	3	3	41.0

SQL ▾

SELECT * FROM Cart;

id	Email
1	josee9425@jmsc.com
2	cousin@jmsc.com
3	maikel@jmsc.com

SELECT * FROM Invoices;

id	Bill_Number	Purchase_Date	Buyer_Email	Buyer_Telephone	Employer_code
1	100001.0	2025-01-25	buyer@jmsc.com	6440-5390	EMP001
2	100002.0	2025-03-15	customer@jmsc.com	8661-1733	EMP002
3	100003.0	2025-12-25	client@jmsc.com	6882-2944	EMP003

SELECT * FROM Products;

id	Product_Code	Name	Price	Entry_Date	Brand	Stock
1	B001	The Great Gatsby	10.99	2023-01-15	Scribner	100
2	B002	1984	12.99	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	15.99	2023-01-17	Tor Books	200

3. Si existe alguna limitante por SQLite, documéntela y resuelva la limitante como considere.

Si al crear la tabla se utiliza una coma en la última columna, da un error “near”.

El mismo error se presenta si no se cierran paréntesis como en el caso de VARCHAR(100.

3 Utilizando el comando ALTER, modifique la tabla de Facturas y agregue una columna para almacenar también el número de teléfono del comprador, y otra para el código de empleado del cajero que realizó la venta.

The screenshot shows a SQLite IDE with the following SQL command in the editor:

```
ALTER TABLE Invoices  
ADD COLUMN Buyer_Telephone INT;
```

The results pane displays the updated 'Invoices' table with the following data:

id	Bill_Number	Purchase_Date	Buyer_Email	Buyer_Telephone	Employer_code
1	100001.0	2025-01-25	buyer@jmsc.com	6440-5390	EMP001
2	100002.0	2025-03-15	customer@jmsc.com	8661-1733	EMP002
3	100003.0	2025-12-25	client@jmsc.com	6882-2944	EMP003

4. Realice los siguientes SELECT:

1. Obtenga todos los productos almacenados:

The screenshot shows a SQLite IDE with the following SQL command in the editor:

```
SELECT *  
FROM Products;
```

The results pane displays the 'Products' table with the following data:

id	Product_Code	Name	Price	Entry_Date	Brand	Stock
1	B001	The Great Gatsby	10000.0	2023-01-15	Scribner	100
2	B002	1984	4000.0	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	15000.0	2023-01-17	Tor Books	200

2. Obtenga todos los productos que tengan un precio mayor a 50000:

The screenshot shows a SQLite IDE with the following SQL command in the editor:

```
SELECT * FROM Products  
WHERE Price > 50000;
```

The results pane displays the 'Products' table with the following data:

id	Product_Code	Name	Price	Entry_Date	Brand	Stock
2	B002	1984	80000.0	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	100000.0	2023-01-17	Tor Books	200

3. Obtenga todas las compras de un mismo producto por id.

The screenshot shows a SQL query in a dark-themed editor: `SELECT Product_id, COUNT(id) FROM Invoice_Products GROUP BY Product_id;`. To the right, the SQL tool's interface shows the same query and a result table with 3 rows and 2 columns: Product_id and COUNT(id). The results are: (1, 1), (2, 1), and (3, 2).

Product_id	COUNT(id)
1	1
2	1
3	2

4. Obtenga todas las compras agrupadas por producto, donde se muestre el total comprado entre todas las compras.

UNION TOTAL--- se utiliza para unir ambos comandos

The screenshot shows a SQL query using UNION ALL: `SELECT Invoice_id, SUM(Total_Amount) as Total_Amount FROM Invoice_Products GROUP BY Invoice_id UNION ALL SELECT "GRAN TOTAL", SUM(Total_Amount) FROM Invoice_Products;`. The result table has 4 rows and 2 columns: Invoice_id and Total_Amount. The results are: (1, 60000.0), (2, 70000.0), (3, 20000.0), and (GRAN TOTAL, 96000.0).

Invoice_id	Total_Amount
1	60000.0
2	70000.0
3	20000.0
GRAN TOTAL	96000.0

5. Obtenga todas las facturas realizadas por el mismo comprador

Se utiliza JOIN para unir la tabla de Cart con la de Invoice_Product y se le indica que Invoice_id va ser igual al id de Cart.

The screenshot shows a SQL query using JOIN: `SELECT Cart.Email, Invoice_Products.Invoice_id, Invoice_Products.Total_Amount FROM Invoice_Products JOIN Cart ON Invoice_Products.Invoice_id = Cart.id ORDER BY Cart.Email;`. The result table has 4 rows and 3 columns: Email, Invoice_id, and Total_Amount. The results are: (cousin@jmsc.com, 2, 40000.0), (cousin@jmsc.com, 2, 30000.0), (josee9425@jmsc.com, 1, 60000.0), and (maikel@jmsc.com, 3, 20000.0).

Email	Invoice_id	Total_Amount
cousin@jmsc.com	2	40000.0
cousin@jmsc.com	2	30000.0
josee9425@jmsc.com	1	60000.0
maikel@jmsc.com	3	20000.0

6. Obtenga todas las facturas ordenadas por monto total de forma descendente

The screenshot shows a SQL query: `SELECT * FROM Invoice_Products ORDER BY Total_Amount DESC;`. The result table has 5 rows and 5 columns: id, Invoice_id, Product_id, Quantity, and Total_Amount. The results are ordered by Total_Amount in descending order: (2, 2, 2, 5, 40000.0), (4, 2, 3, 3, 30000.0), (3, 3, 3, 2, 20000.0), and (1, 1, 1, 2, 60000.0).

id	Invoice_id	Product_id	Quantity	Total_Amount
2	2	2	5	40000.0
4	2	3	3	30000.0
3	3	3	2	20000.0
1	1	1	2	60000.0

7. Obtenga una sola factura por número de factura.

```
SELECT Bill_Number FROM Invoices
WHERE Bill_Number = 100003;
```

SQL ▾

SELECT Bill_Number FROM Invoices
WHERE Bill_Number = 100003;

Bill_Number
100003.0